# Design Patterns 1: clasificare, Singleton, Visitor, N-tier archit., Inversion of Control

Alexandru Olteanu

Universitatea Politehnica Bucuresti
Facultatea de Automatică si Calculatoare, Departamentul Calculatoare
alexandru.olteanu@cs.pub.ro

OOP, 2018



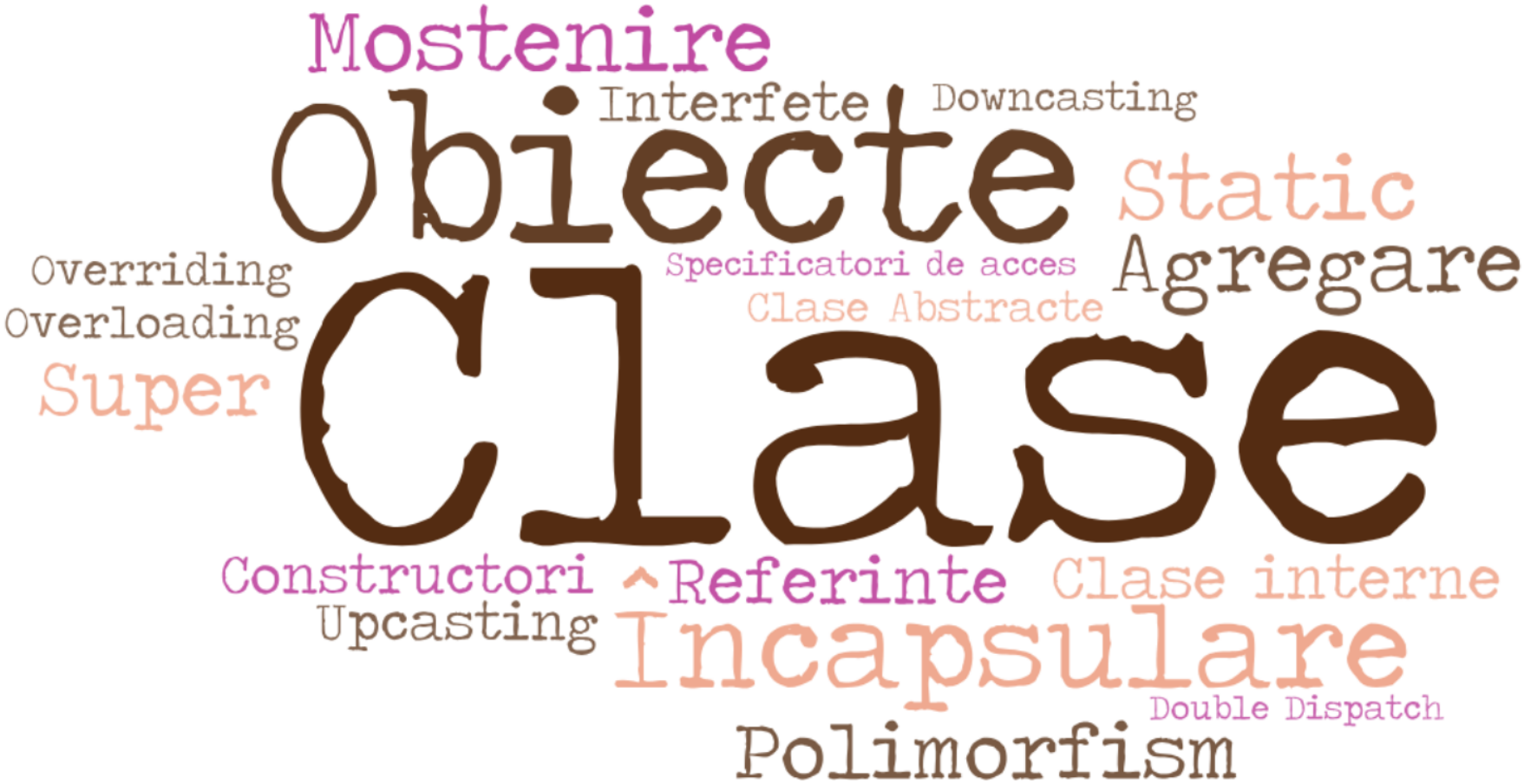




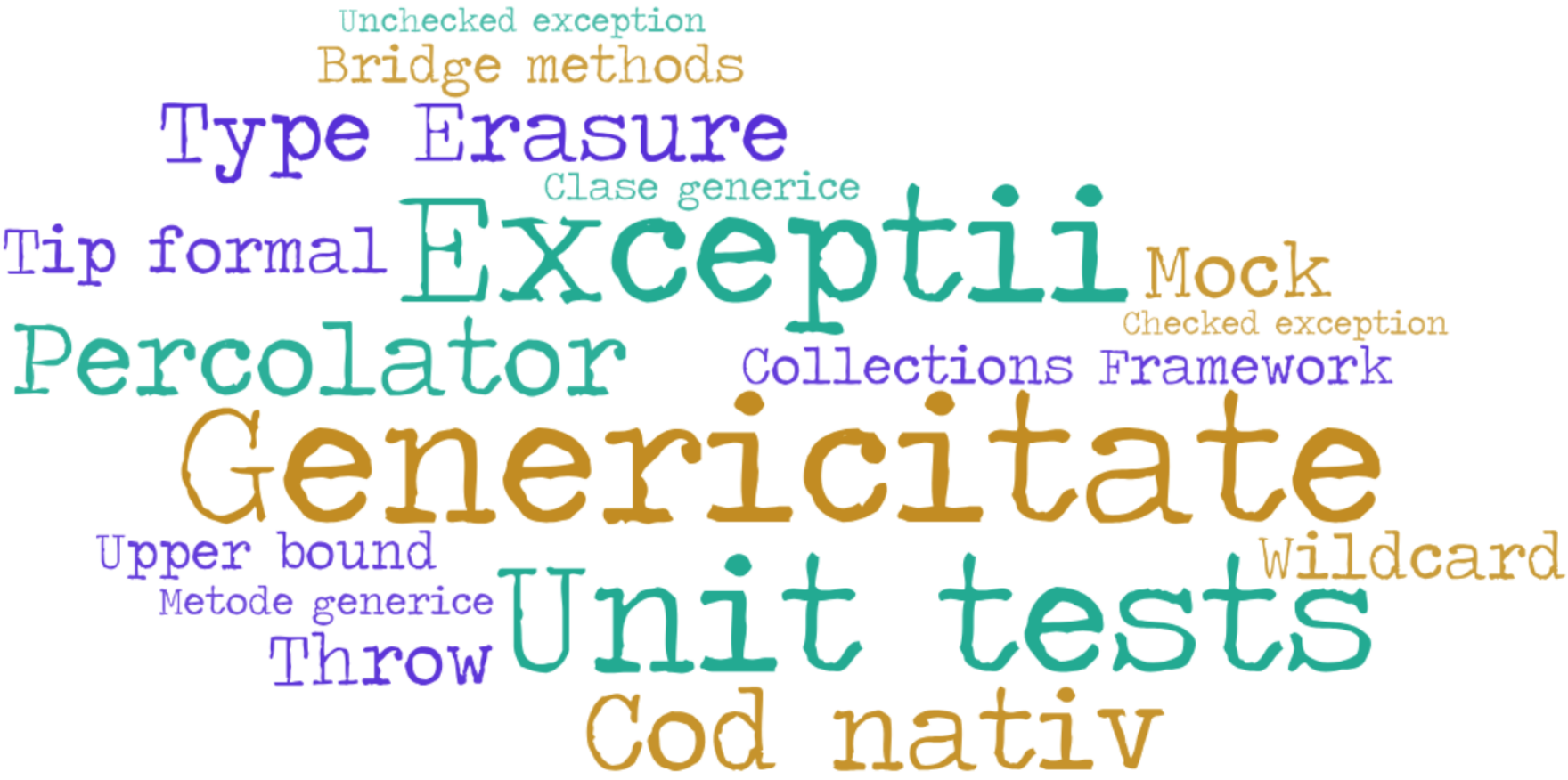



Universitatea
Politehnica
București

# Partea 1: Bazele teoretice ale OOP



# Partea a 2-a: Antract pragmatic



# Design Patterns

# Limitari si potential in POO

- cum abordam o problema, astfel incat sa o gandim intai la nivel abstract, nu de detaliu (desi acesta ar fi mai accesibil)?
- cum sa ne ferim de ▸ Class Explosion ?
- cum sa ne ferim de a avea cod duplicat?
- cum sa proiectam clase intr-un mod corect, logic si usor de intretinut?
- sigur a mai avut cineva o problema similara de rezolvat; cum a facut?

# Ce sunt Design Patterns?

Each pattern describes a problem that occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.
- *Christopher Alexander*

# Ce sunt Design Patterns?

Each pattern describes a problem that occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.
- *Christopher Alexander*

e.g. Courtyard

# Why use them?

- reuse solutions
- common terminology
- shift mindset

# Clasificare Design Patterns

- *creational patterns*: definesc mecanisme de creare a obiectelor (GoF)
- *structural patterns*: definesc relații între entități (GoF)
- *behavioral patterns*: definesc comunicarea între entități (GoF)
- *concurrency patterns*: definesc mecanisme utile in programarea pe paralela si distribuita (sincronizare etc.)
- *architectural patterns*: descriu structura intregului sistem (e.g. MVC, multi-tier etc.)

# GoF Design Patterns

E. Gamma, R. Helm, R. Johnson, J. Vlissides (aka Gang of Four)
"Design Patterns: Elements of Reusable Object-Oriented Software"

| Creational | Structural | Behavioral |
|---|---|---|
| Abstract Factory<br>Builder<br>Prototype<br>Singleton | Adapter<br>Bridge<br>Composite<br>Decorator<br>Facade<br>Flyweight<br>Proxy | Chain of Responsibility<br>Command<br>Interpreter<br>Iterator<br>Mediator<br>Memento<br>Observer<br>State<br>Strategy<br>Visitor |

# What's in a Design Pattern

- name
- why? problem that the pattern solves
- how? solution structure, implementation examples
- what? consequences

# How to use a Design Pattern

- Read the pattern for an overview
- Choose names for the pattern participants that are meaningful in the application context
- Define interfaces, classes and relationships between them, or alter existing ones accordingly
- Define and implement operations giving application specific names

# Singleton

# Description

Singleton ensures a class only has one instance, and provides a global point of access to it.

# Problem that the pattern solves

- uneori este nevoie sa avem o singura instanta a unei clase

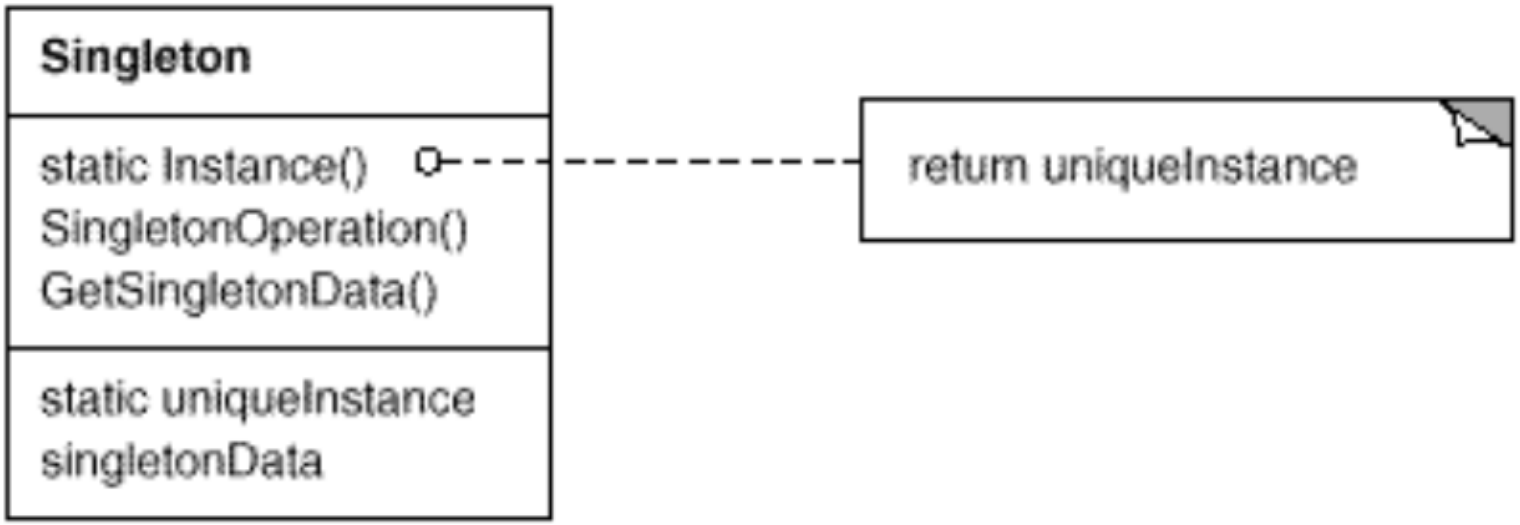
# Problem that the pattern solves

- uneori este nevoie sa avem o singura instanta a unei clase
- ar trebui sa ne putem razgandi relativ usor

# Problem that the pattern solves

- uneori este nevoie sa avem o singura instanta a unei clase
- ar trebui sa ne putem razgandi relativ usor
- trebuie sa fie clar pentru oricine foloseste codul nostru cum poate obtine acea instanta

# Solution Structure




* E. Gamma, R. Helm, R. Johnson, J. Vlissides "Design Patterns: Elements of Reusable Object-Oriented Software"

# Solution Details

- clasa incapsuleaza unica sa instanta si gestioneaza accesul la ea

# Solution Details

- clasa incapsuleaza unica sa instanta si gestioneaza accesul la ea
- exista alternativa de a folosi metode statice, dar este mai rigida (in C++ nu pot fi extinse, in general mai greu sa ne razgandim)

# Implementation Examples

```
public class ClassicSingleton {

    private static ClassicSingleton instance = null;
    private ClassicSingleton() {
        // Exists only to defeat instantiation.
    }

    public static ClassicSingleton getInstance() {
        if(instance == null) {
            instance = new ClassicSingleton();
        }
        return instance;
    }
}

ClassicSingleton c = ClassicSingleton.getInstance();
```

# Exercitii

- gasiti in codul ▸ EduMediaPlayer: o clasa care ar avea sens sa fie implementata ca Singleton
- ganditi-va daca exista vreo modalitate de a pacali mecanismul si a obtine mai multe instante

# Consequences

- acces controlat la unica instanta

# Consequences

- acces controlat la unica instanta
- evita poluarea cu variabile globale

# Consequences

- acces controlat la unica instanta
- evita poluarea cu variabile globale
- clasa poate fi rafinata simplu prin extindere

# Consequences

- acces controlat la unica instanta
- evita poluarea cu variabile globale
- clasa poate fi rafinata simplu prin extindere
- e usor sa ne razgandim si sa permitem mai multe instante, chiar putem pune o limita a numarului de instante

# Consequences

- acces controlat la unica instanta
- evita poluarea cu variabile globale
- clasa poate fi rafinata simplu prin extindere
- e usor sa ne razgandim si sa permitem mai multe instante, chiar putem pune o limita a numarului de instante
- mai flexibila decat alternativa de a folosi metode statice

# Visitor

# Description

Visitor represents an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates.

# Problem that the pattern solves

- o structura contine obiecte de clase variate, asupra carora vrem sa aplicam operatii ce depind de clasele lor concrete

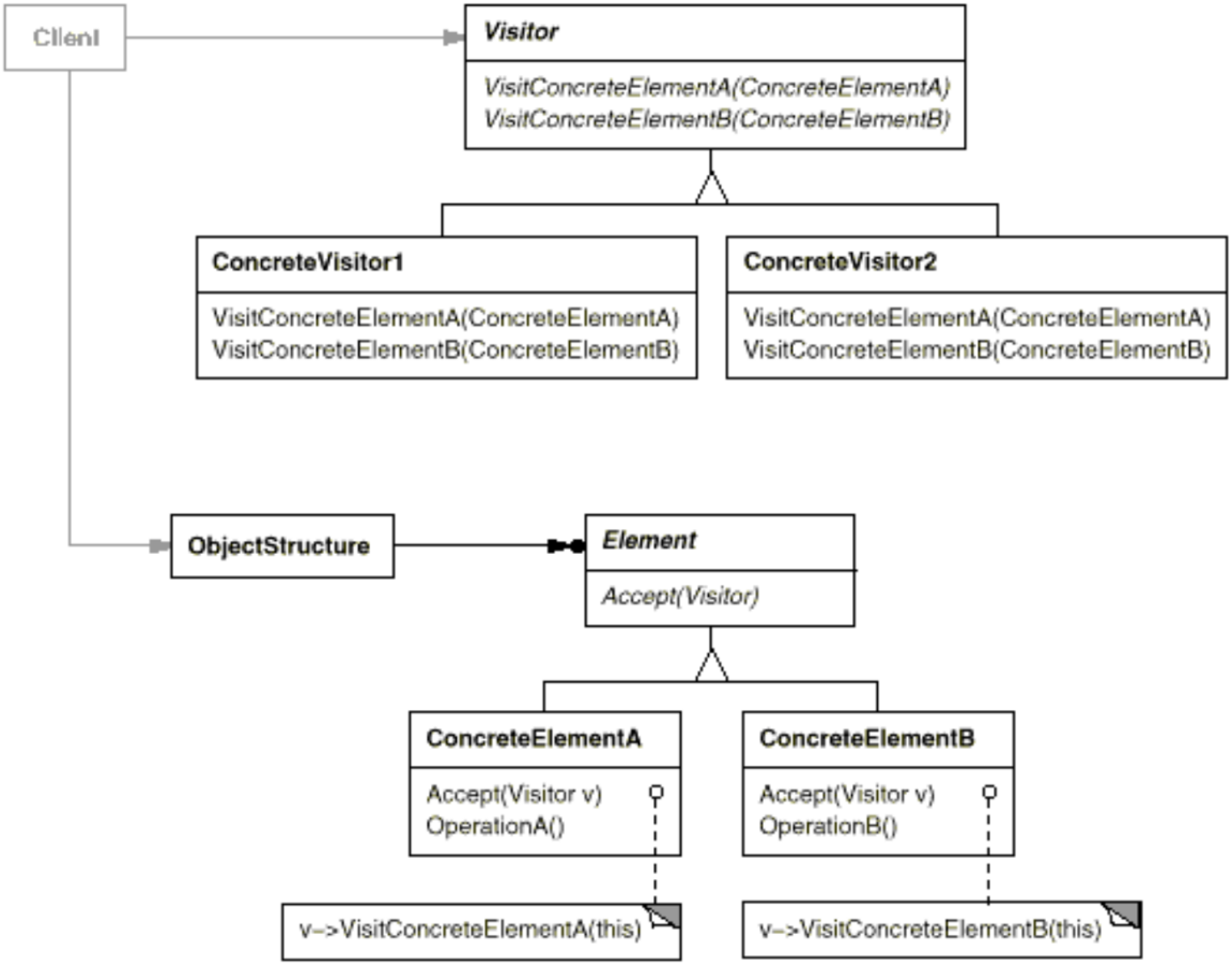
# Problem that the pattern solves

- o structura contine obiecte de clase variate, asupra carora vrem sa aplicam operatii ce depind de clasele lor concrete
- operatiile pe care vrem sa le aplicam sunt diferite, fara legatura intre ele si ar trebui sa fie usor extensibile

# Problem that the pattern solves

- o structura contine obiecte de clase variate, asupra carora vrem sa aplicam operatii ce depind de clasele lor concrete
- operatiile pe care vrem sa le aplicam sunt diferite, fara legatura intre ele si ar trebui sa fie usor extensibile
- clasele ce definesc structura de obiecte se schimba rar, dar vrem sa definim operatii noi usor (daca si clasele ce definesc structura de obiecte se schimba des, poate e mai bine sa definim operatiile in ele)

# Solution Structure



# Solution Details

Un client trebuie sa instantieze cate un vizitator concret pentru fiecare operatie si sa traverseze structura de obiecte vizitand fiecare obiect cu fiecare vizitator concret.

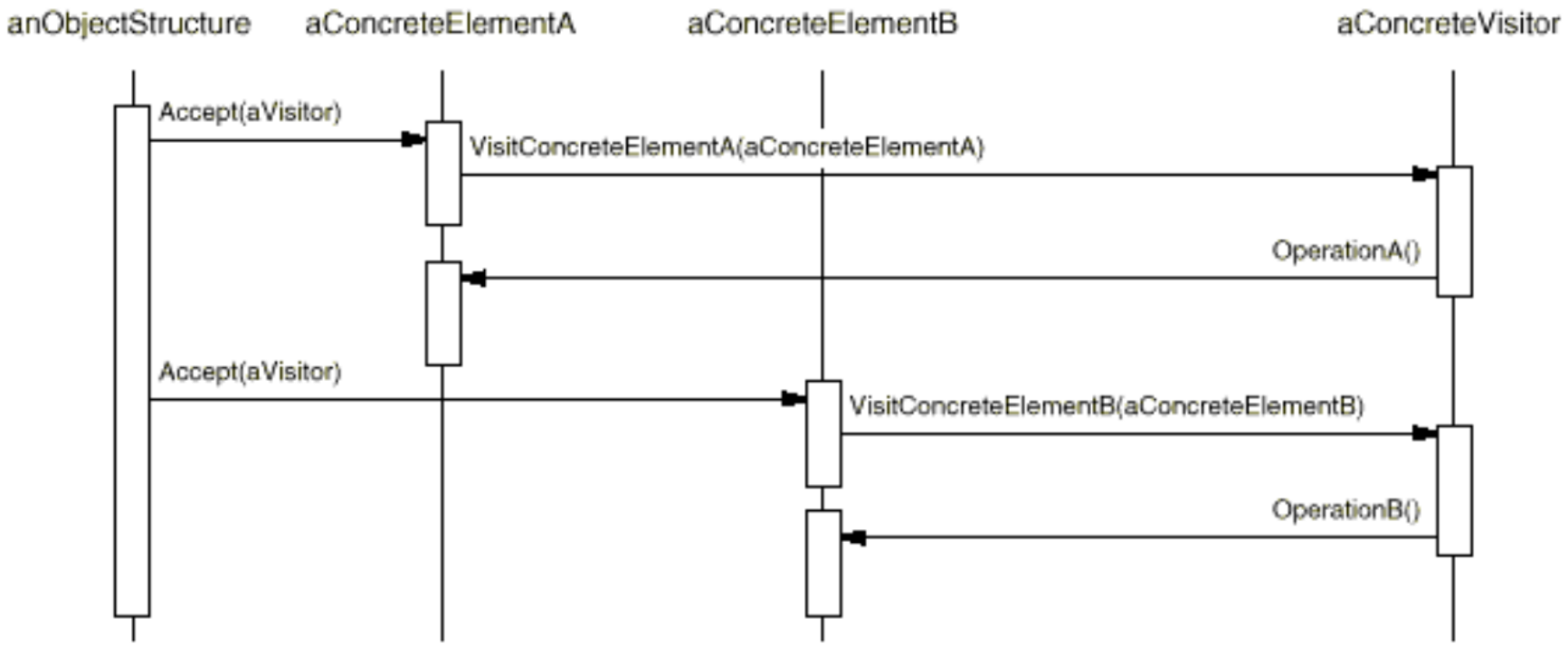
# Solution Details

Un client trebuie sa instantieze cate un vizitator concret pentru fiecare operatie si sa traverseze structura de obiecte vizitand fiecare obiect cu fiecare vizitator concret.
Cum se face traversarea?

- de catre structura de obiecte
- de catre vizitator
- intr-un obiect iterator separat

# Solution Details

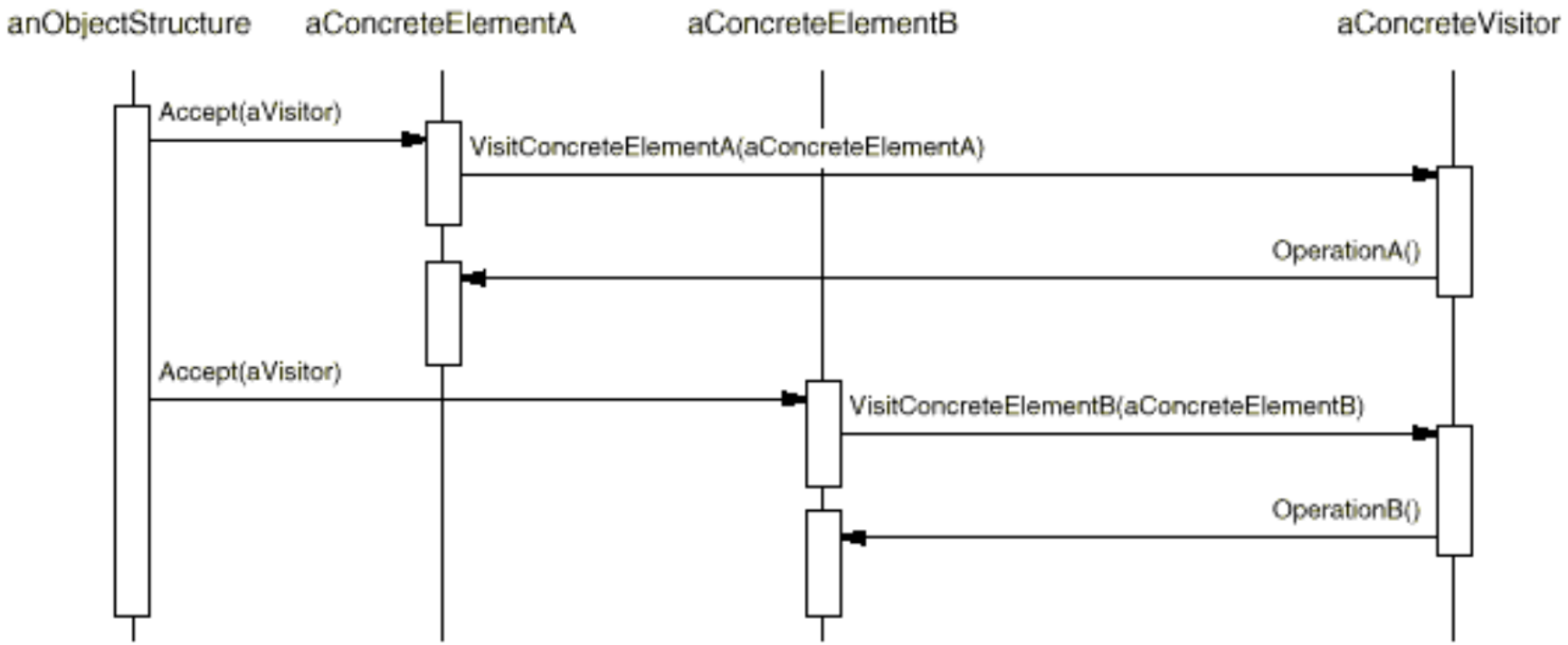
Vizitatorul concret ofera un context in care sa se desfasoare operatia, acumuleaza stare si poate apela metode din obiecte:




* E. Gamma, R. Helm, R. Johnson, J. Vlissides "Design Patterns: Elements of Reusable Object-Oriented Software"

# Solution Details

Vizitatorul concret ofera un context in care sa se desfasoare operatia, acumuleaza stare si poate apela metode din obiecte:




* E. Gamma, R. Helm, R. Johnson, J. Vlissides "Design Patterns: Elements of Reusable Object-Oriented Software"

Exista un echilibru care variaza de la o solutie la alta intre ce se desfasoara in vizitator si ce este expus de obiect prin metode

# Solution Details

Double Dispatch inseamna ca o actiune (precum Accept) depinde de tipul a doua elemente (in cazul Accept depinde de tipul vizitatorului si al obiectului vizitat).

# Solution Details

Double Dispatch inseamna ca o actiune (precum Accept) depinde de tipul a doua elemente (in cazul Accept depinde de tipul vizitatorului si al obiectului vizitat).

| | ConcreteVisitor1 | ConcreteVisitor2 |
|---|---|---|
| ConcreteElementA | VisitConcreteElementA | VisitConcreteElementA |
| ConcreteElementB | VisitConcreteElementB | VisitConcreteElementB |

# Exercitii

- Incercati sa adaugati o noua clasa de vizitator
- Incercati sa adaugati o noua clasa de obiecte vizitate

# Consequences

- adaugarea de operatii este simpla (adaugarea unui nou vizitator vs modificarea fiecare clase de obiecte vizitate)

# Consequences

- adaugarea de operatii este simpla (adaugarea unui nou vizitator vs modificarea fiecare clase de obiecte vizitate)
- vizitatorul grupeaza operatiile intr-un mod logic

# Consequences

- adaugarea de operatii este simpla (adaugarea unui nou vizitator vs modificarea fiecare clase de obiecte vizitate)
- vizitatorul grupeaza operatiile intr-un mod logic
- adaugarea de noi clase de obiecte vizitate este mai dificila

# Consequences

- adaugarea de operatii este simpla (adaugarea unui nou vizitator vs modificarea fiecare clase de obiecte vizitate)
- vizitatorul grupeaza operatiile intr-un mod logic
- adaugarea de noi clase de obiecte vizitate este mai dificila
- vizitarea structurii de obiecte vizitate se poate face in diferite moduri, dar in forma cea mai pura de Visitor, spre deosebire de Iterator, obiectele vizitate nu trebuie neaparat sa instantieze dintr-o ierarhie de clase

# Consequences

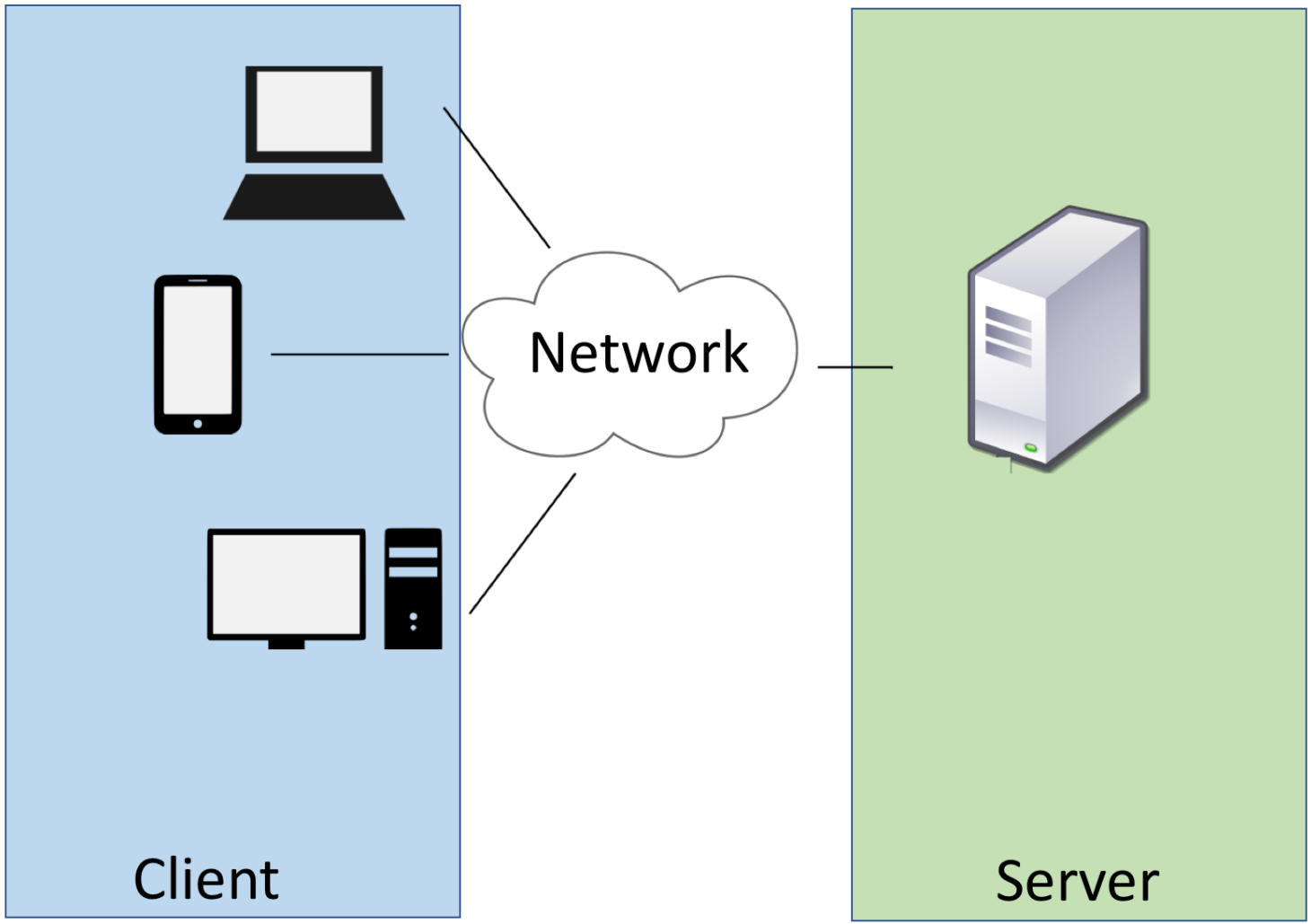
- adaugarea de operatii este simpla (adaugarea unui nou vizitator vs modificarea fiecare clase de obiecte vizitate)
- vizitatorul grupeaza operatiile intr-un mod logic
- adaugarea de noi clase de obiecte vizitate este mai dificila
- vizitarea structurii de obiecte vizitate se poate face in diferite moduri, dar in forma cea mai pura de Visitor, spre deosebire de Iterator, obiectele vizitate nu trebuie neaparat sa instantieze dintr-o ierarhie de clase
- vizitatorul poate acumula stare

# Consequences

- adaugarea de operatii este simpla (adaugarea unui nou vizitator vs modificarea fiecare clase de obiecte vizitate)
- vizitatorul grupeaza operatiile intr-un mod logic
- adaugarea de noi clase de obiecte vizitate este mai dificila
- vizitarea structurii de obiecte vizitate se poate face in diferite moduri, dar in forma cea mai pura de Visitor, spre deosebire de Iterator, obiectele vizitate nu trebuie neaparat sa instantieze dintr-o ierarhie de clase
- vizitatorul poate acumula stare
- implementarea de operatii in obiectele vizitate care sa permita vizitatorului sa isi faca treaba poate compromite incapsularea in clasele de obiecte vizitate

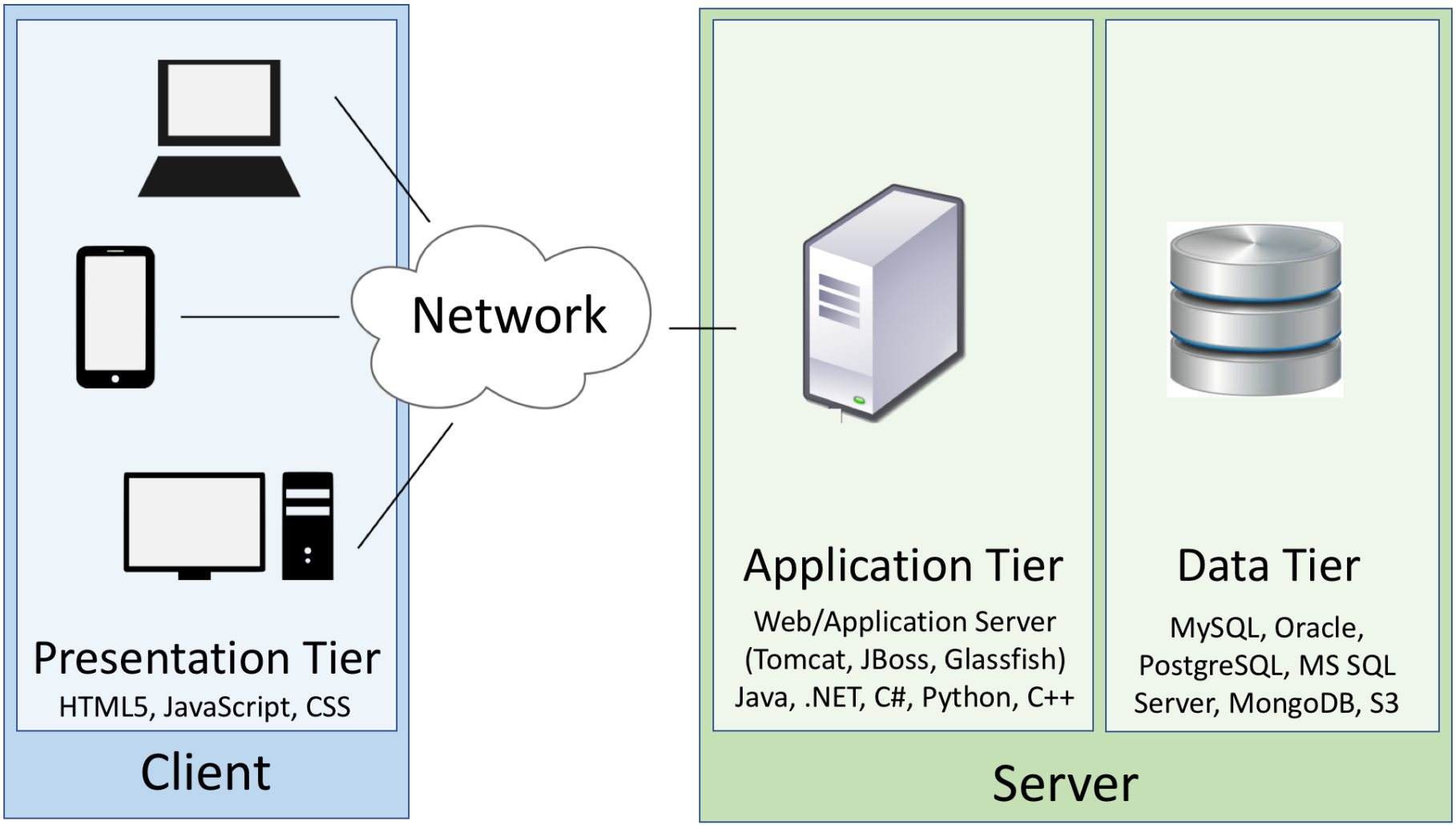
# Architectural Patterns

# Client-server model

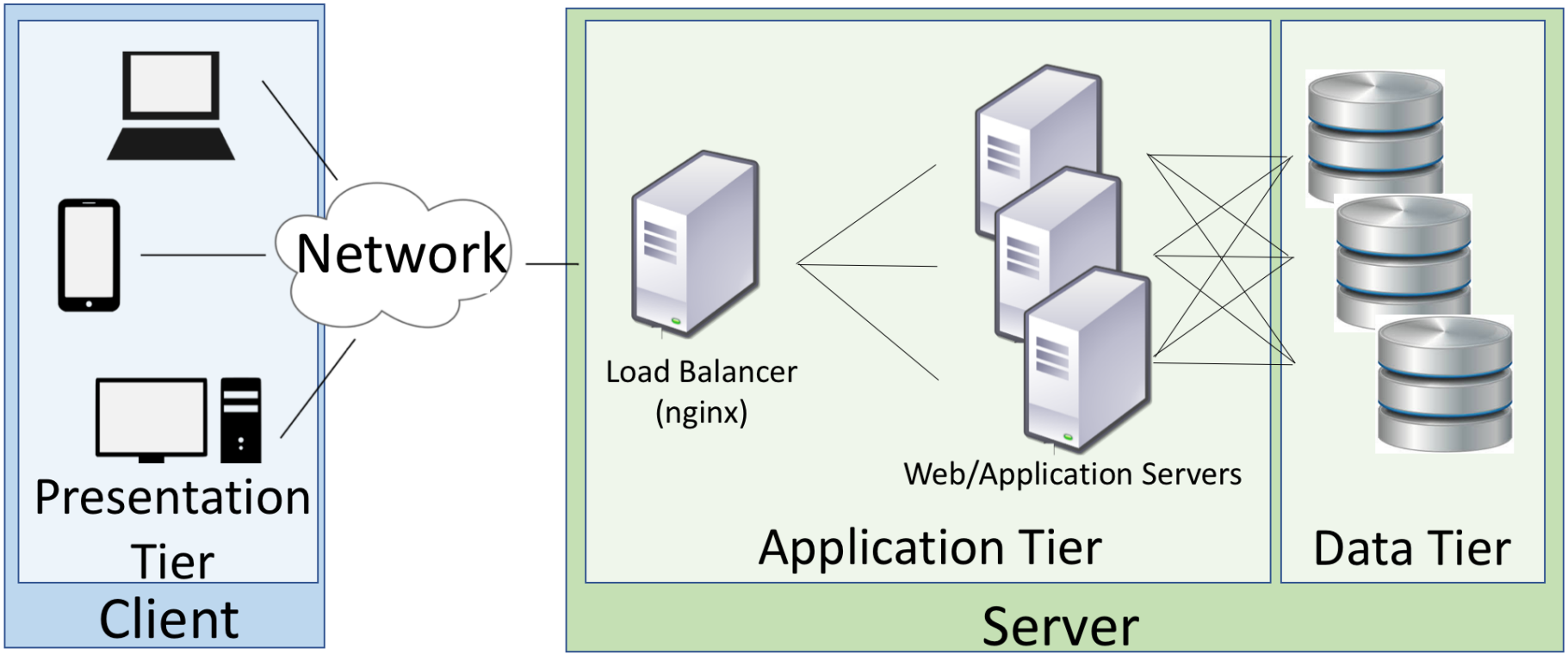



as opposed to peer-to-peer model

# Three-Tier Architecture



# Three-Tier Architecture

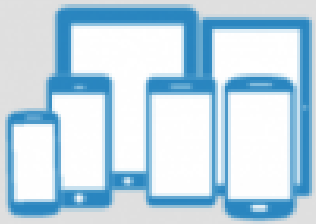



- Reliability and Availability
- Scalability
- Extensibility and Maintanibility

▸ More info

# Four-Tier Architecture

**Client tier**



- Mobile clients
- Wearables
- Internet of things
- Responsible for experience delivery

**Delivery tier**



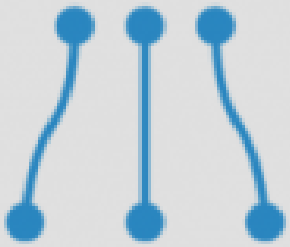
- Optimizes content for proper display on device
- Caches content for performant delivery
- Drives personalization by using analytics to monitor user behavior

**Aggregation tier**



- Aggregates and federates services tier data
- Provides discovery for the underlying service library
- Performs data protocol translation (e.g., SOAP to JSON)

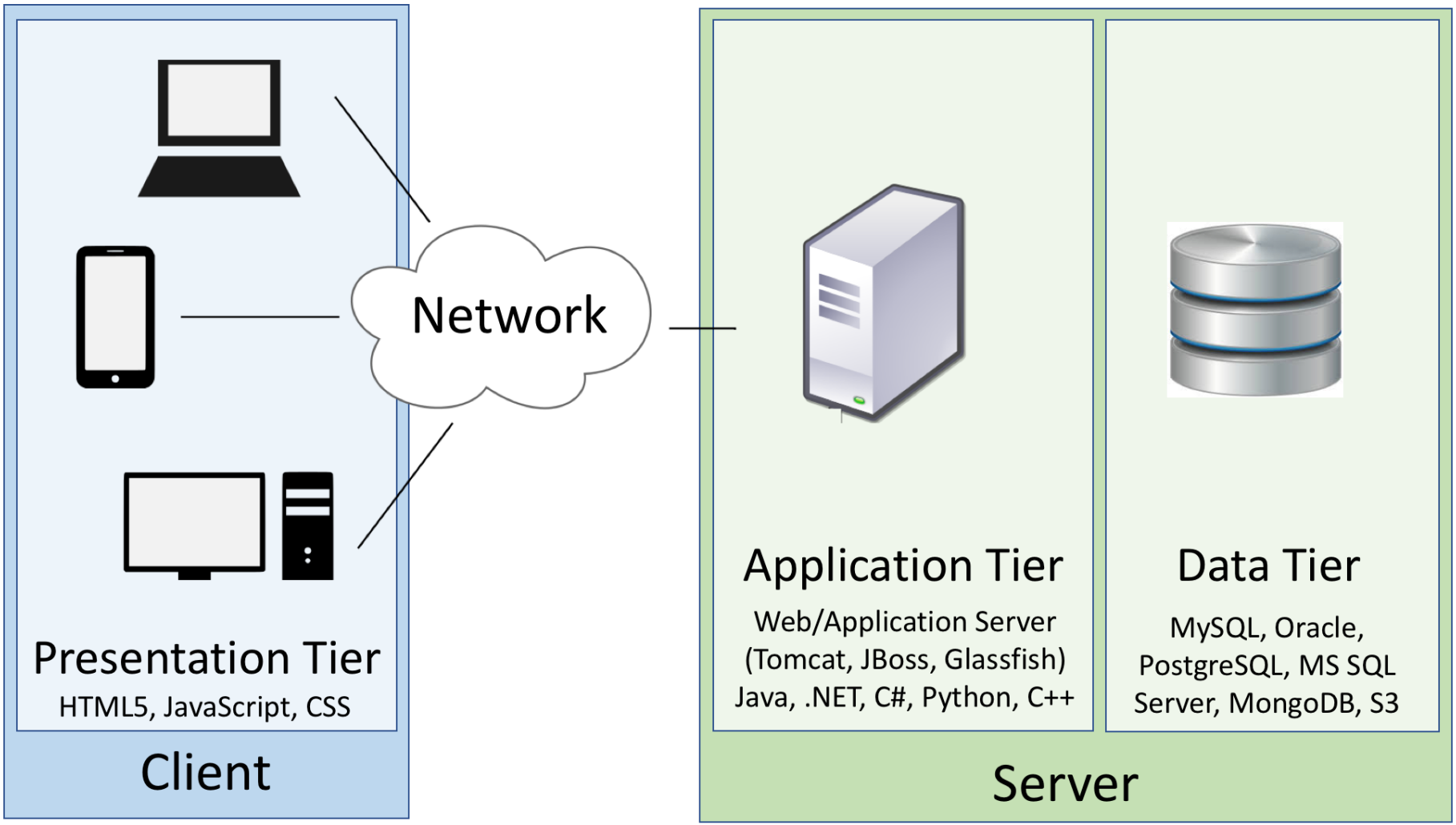
**Services tier**



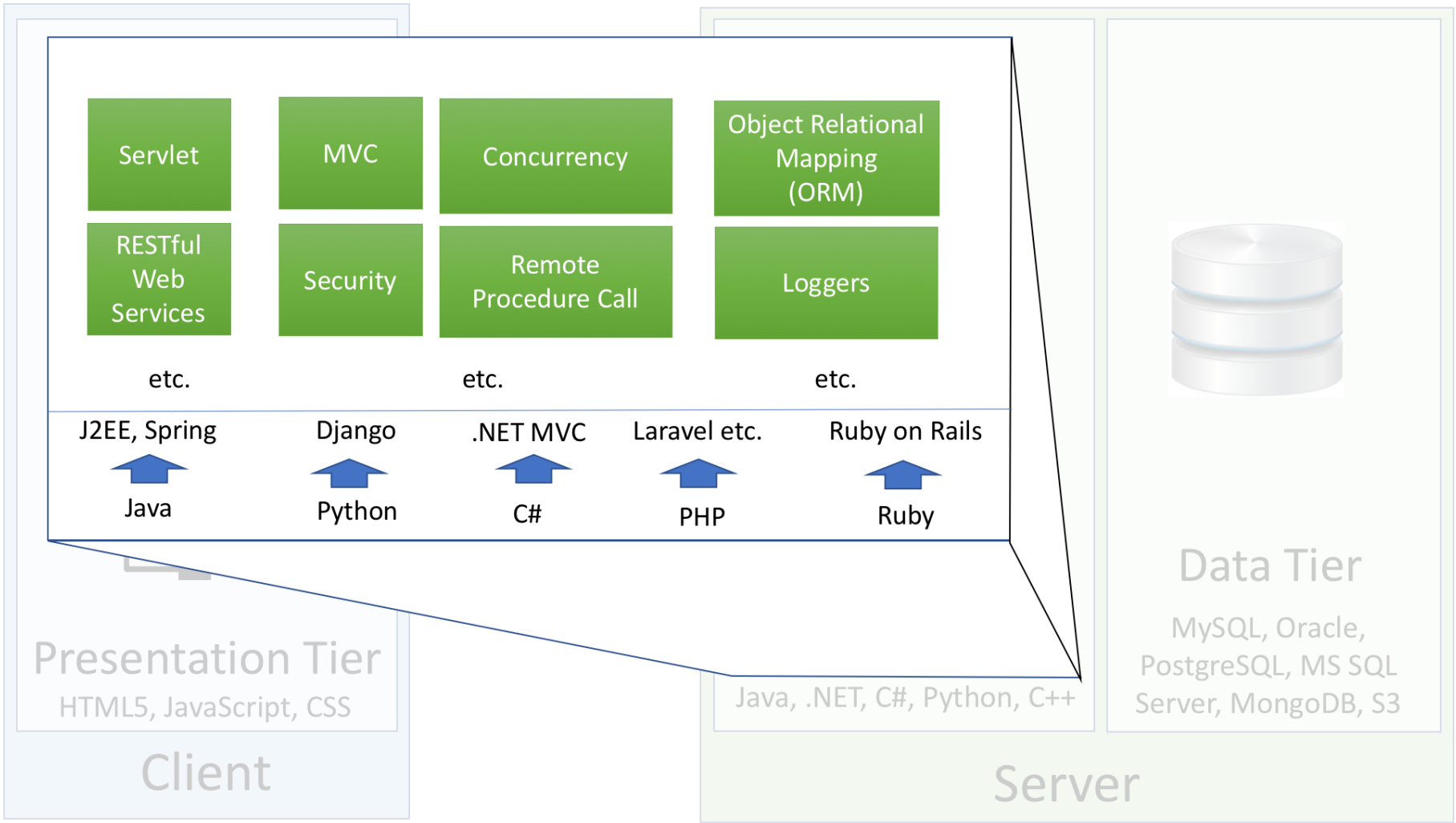
- Existing on-premises systems of record, services, and data
- External third-party services (e.g., Box, Twilio, Urban Airship)

▸ It's Time to Move to a Four-Tier Application Architecture, Nginx

# Three-Tier Architecture



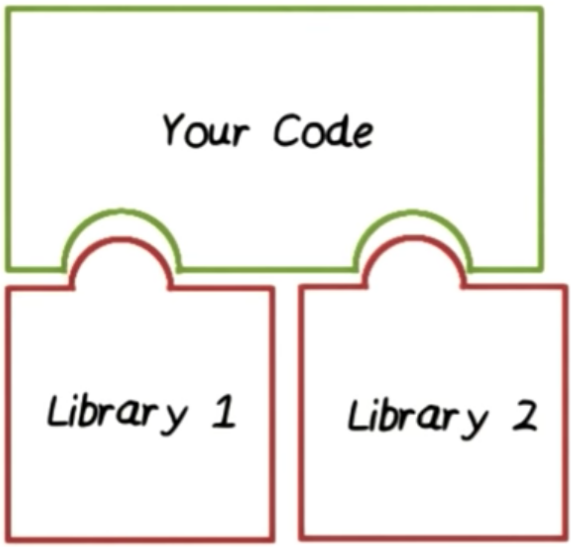
# Frameworks

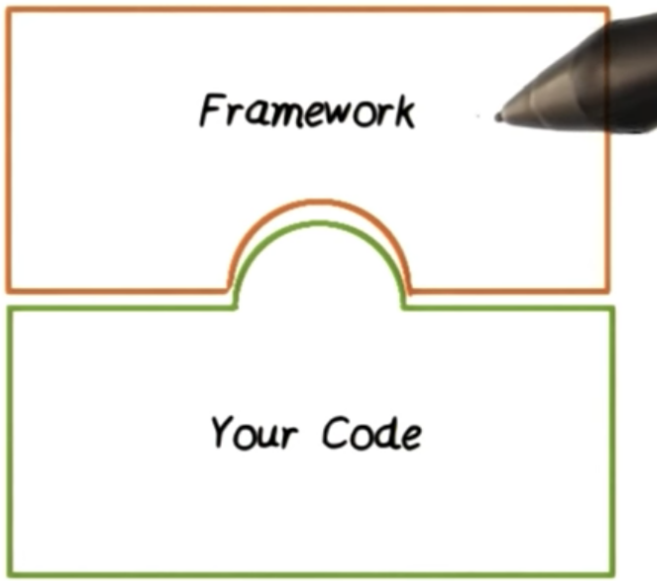

Servlet
MVC
Concurrency
Object Relational Mapping (ORM)
RESTful Web Services
Security
Remote Procedure Call
Loggers
etc.
etc.
etc.
J2EE, Spring
Django
.NET MVC
Laravel etc.
Ruby on Rails
Java
Python
C#
PHP
Ruby
Presentation Tier
HTML5, JavaScript, CSS
Client
Java, .NET, C#, Python, C++
Server
Data Tier
MySQL, Oracle, PostgreSQL, MS SQL Server, MongoDB, S3

# Inversion of Control

## Definition

Inversion of Control is a design principle in which custom code you write receives flow of control from a generic framework (as opposed to a 'traditional' architecture where custom code you write is the one that calls into reusable libraries)




▸ Inversion of Control Youtube video

# Inversion of Control: implementation techniques

Many implementation techniques, relying heavily on design patterns:

- Service Locator pattern
- Dependency Injection
- Contextualized lookup
- Template Method pattern
- Strategy pattern

▸ Inversion of Control on Wikipedia

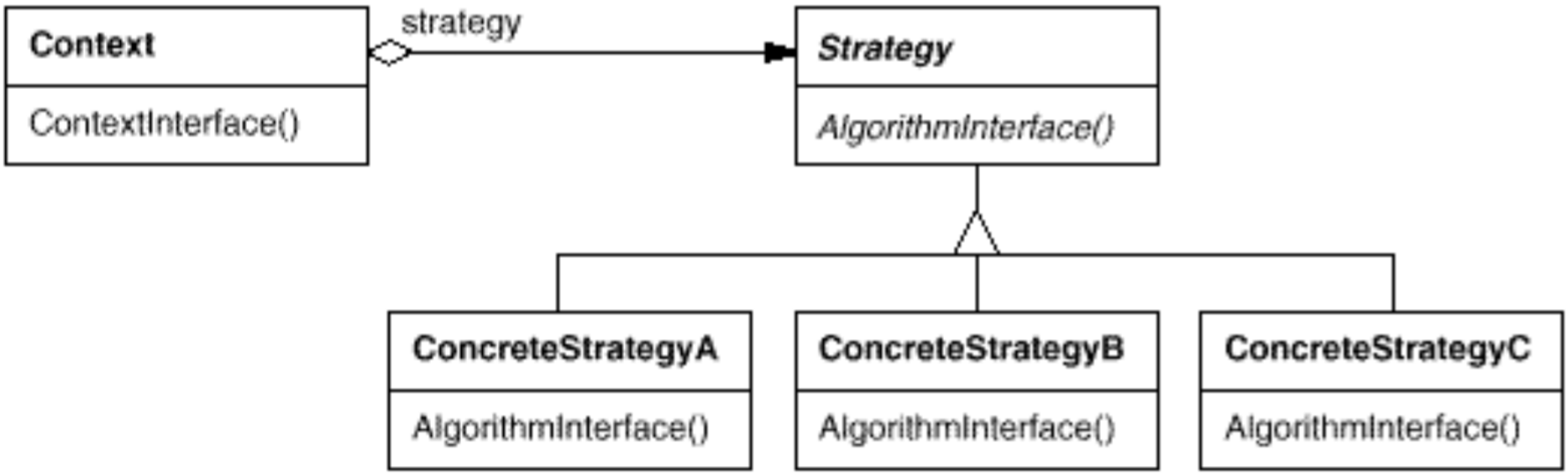
# Strategy

Strategy defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it

- vrem sa putem alege la runtime varianta de algoritm cea mai potrivita, sau varianta cea mai potrivita depinde de multe conditii
- vrem sa nu ingreunam o clasa cu implementari de algoritmi

e.g. Collections#sort() apeleaza java.util.Comparator#compare()

# Solution




* E. Gamma, R. Helm, R. Johnson, J. Vlissides "Design Patterns: Elements of Reusable Object-Oriented Software"

- Strategy abstractizeaza toti algoritmii suportati
- Context este configurat cu un ConcreteStrategy sau poate alege un ConcreteStrategy si poate expune modalitati prin care Strategy sa acceseze date pentru a rula algoritmii

# Dependency Injection

## Definition

Dependencies are other classes that your class needs to function.

# Dependency Injection

## Definition

Dependencies are other classes that your class needs to function.

## Definition

Dependency Injection is a design pattern that:

- requires custom classes to link to Dependencies through setters or constructors (instead of instantiating with new)
- allows frameworks to inject proper implementations to those Dependencies (aka autowiring in some frameworks)
- heavily relies on the Dependency Inversion Principle (remember SOLID)

▸ Dependency Injection Youtube video

# Spring Boot

Hello World in Spring Boot ▸ see EduRenewableEnergy

- lambda expressions (can be seen as anonymous methods) ▸ see docs
- annotations ▸ @Bean ▸ @SpringBootApplication
- Dependency Injection
- GoF design patterns - cate puteti recunoaste?

# Reading Material

# Reading Material

- Intro level:
  Design Patterns Explained
  A. Shalloway, Addison Wesley, 2002.
- Professional level:
  Design Patterns: Elements of Reusable Object-Oriented Software.
  E. Gamma, R. Helm, R. Johnson, J. Vlissides, Addison Wesley, 1994.
- Cultura Generala:
  The Timeless Way of Building
  C. Alexander, Oxford University Press, 1977.

# De citit

## Este util sa cititi textul laboratorului inainte de laborator:

- Lab 9: JUnit si Exceptii